

CC-ADOV: An Effective Multiple Paths Congestion Control AODV

Student Name: Dipit Saha

Roll Number: 2105050

June 5, 2026

Contents

1	Introduction	3
1.1	Background	3
1.2	Motivation	3
1.3	Project Scope	3
2	Literature Review	3
2.1	Paper 1: CC-AODV (Base Paper)	3
2.1.1	Key Contributions	4
2.1.2	How CC-AODV Works	4
3	Problem Statement	4
3.1	Current Problems in CC-AODV	4
4	Proposed Solution: ECC-AODV	5
4.1	Overview	5
4.2	Modification 1: Adaptive Threshold Management (ATM)	6
4.2.1	Concept	6
4.2.2	Benefits	6
4.3	Modification 2: Queue-Aware Congestion Detection (QACD)	6
4.3.1	Concept	6
4.3.2	Congestion Level Formula	6
4.3.3	Decision Making	7
4.4	Modification 3: Optimized Packet Header Design (OPHD)	7
4.4.1	Problem	7
4.4.2	Solution	7
4.5	Modification 4: Multi-Metric Path Selection (MMPS)	7
4.5.1	Concept	7
4.5.2	Path Score Formula	8
4.5.3	Algorithm	8
4.6	Complete ECC-AODV Algorithm Flow	8
5	Challenges and Mitigation	8
5.1	Technical Challenges	8
6	Future Work	10
6.1	Potential Extensions	10

1 Introduction

1.1 Background

Mobile Ad hoc Networks (MANETs) are self-configuring networks of mobile devices connected wirelessly without requiring fixed infrastructure. These networks find applications in disaster relief, military operations, vehicular networks, and emergency response systems where traditional infrastructure is unavailable or impractical.

The Ad hoc On-Demand Distance Vector (AODV) routing protocol is one of the most widely deployed reactive routing protocols in MANETs. However, traditional AODV suffers from performance degradation under network congestion, as it always selects the shortest path without considering the load on intermediate nodes.

1.2 Motivation

Current challenges in MANET routing include:

- **Congestion-blind routing:** Traditional AODV selects paths based solely on hop count, ignoring node congestion levels
- **Resource underutilization:** Some nodes remain idle while others are overloaded
- **Increased packet loss:** Congested nodes drop packets, degrading network performance
- **Higher delays:** Packets queue at busy nodes, increasing end-to-end delay
- **Poor scalability:** Performance degrades significantly as network density increases

1.3 Project Scope

This project focuses on enhancing the CC-AODV protocol to address its current limitations while maintaining backward compatibility with AODV and improving overall network performance.

2 Literature Review

2.1 Paper 1: CC-AODV (Base Paper)

Title: CC-AODV: An Effective Multiple Paths Congestion Control AODV

Authors: Yefa Mai, Fernando Molina Rodriguez, and Dr. Nan Wang

Published: 2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)

Link: <https://ieeexplore.ieee.org/document/8301758>

DOI: 10.1109/CCWC.2018.8301758

2.1.1 Key Contributions

1. Introduced a **congestion counter** mechanism in the AODV routing table
2. Added a **congestion flag** (32-bit) to RREP packet headers
3. Implemented counter-based RREQ dropping at intermediate nodes
4. Demonstrated improved packet delivery ratio and throughput in NS-3 simulations

2.1.2 How CC-AODV Works

1. Route Discovery:

- Source floods RREQ packets across the network
- Intermediate nodes check congestion counter against threshold
- If counter $<$ threshold: forward RREQ and update routing table
- If counter $\geq threshold$: *dropRREQ(pathtoocongested)*

2. Route Reply:

- Destination generates RREP with congestion flag = true
- RREP unicasts back through intermediate nodes
- Each node checks congestion flag
- If flag = true: increment local congestion counter
- Update routing table with new route information

3. Counter Management:

- **Initialize:** Counter = 0 when routing table created
- **Increment:** Counter++ when RREP with congestion flag passes through
- **Decrement:** Counter– when route lifetime expires or RERR sent
- **Reset:** Counter = 0 when node removed from network

3 Problem Statement

3.1 Current Problems in CC-AODV

Despite improvements over vanilla AODV, CC-AODV suffers from several critical limitations:

1. Fixed Threshold Problem:

- Threshold value is hardcoded
- Optimal threshold varies with network density and traffic load
- Fixed threshold performs poorly in dynamic scenarios
- No mechanism to adapt threshold based on network conditions

2. Inefficient Memory Usage:

- Uses 32-bit field for boolean congestion flag (wasteful)
- Adds overhead to every RREP packet (4 bytes per packet)
- No compression or optimization of routing table entries

3. Crude Congestion Detection:

- Only counts number of routes passing through node
- Ignores actual queue length and packet processing rate
- Does not consider buffer occupancy or channel utilization
- Binary flag (congested/not congested) lacks granularity

4. Lack of Path Diversity:

- Still selects single path based on counter comparison
- Does not maintain alternative routes
- No load balancing across multiple available paths
- Cannot adapt if selected path becomes congested mid-transmission

5. Higher End-to-End Delay:

- Selecting longer (less congested) paths increases hop count
- Additional processing overhead from counter checks
- Delay increases significantly in dense networks (25% increase at 50 nodes)

6. Limited Scalability:

- Performance degrades in very dense networks
- Control overhead increases with network size
- No differentiation between traffic types (UDP vs TCP)

4 Proposed Solution: ECC-AODV

4.1 Overview

Enhanced CC-AODV (ECC-AODV) introduces four major improvements:

- 1. Adaptive Threshold Management (ATM)**
- 2. Queue-Aware Congestion Detection (QACD)**
- 3. Optimized Packet Header Design (OPHD)**

4.2 Modification 1: Adaptive Threshold Management (ATM)

4.2.1 Concept

Instead of using a fixed threshold value, ATM dynamically adjusts the congestion threshold based on:

- Current network density (number of active nodes)
- Average congestion level across all nodes
- Packet delivery ratio trends
- Traffic intensity

4.2.2 Benefits

- Adapts to varying network sizes automatically
- Responds to network stress in real-time
- Prevents overly restrictive routing in low-traffic scenarios
- Maintains effectiveness in high-traffic scenarios

4.3 Modification 2: Queue-Aware Congestion Detection (QACD)

4.3.1 Concept

Instead of relying solely on route counter, QACD incorporates actual queue measurements:

- Current queue length (number of packets in buffer)
- Queue occupancy percentage
- Packet drop rate at the queue
- Average waiting time in queue

4.3.2 Congestion Level Formula

$$CL = w_1 \times \frac{Q_{current}}{Q_{max}} + w_2 \times \frac{Counter}{Threshold} + w_3 \times DropRate \quad (1)$$

Where:

- CL = Congestion Level (0 to 1)
- $Q_{current}$ = Current queue length
- Q_{max} = Maximum queue capacity
- $Counter$ = Current congestion counter value
- $Threshold$ = Dynamic threshold from ATM
- $DropRate$ = Recent packet drop rate (0 to 1)
- w_1, w_2, w_3 = Weights (must sum to 1.0)

4.3.3 Decision Making

Algorithm 1 QACD-based Route Decision

```
1:  $CL \leftarrow$  Calculate congestion level using Equation 1
2:
3: if  $CL < 0.3$  then
4:   return LOW CONGESTION                                ▷ Accept and forward RREQ
5: else if  $CL < 0.7$  then
6:   return MODERATE CONGESTION                            ▷ Accept but mark in RREP
7: else
8:   return HIGH CONGESTION                                ▷ Drop RREQ
9: end if
```

4.4 Modification 3: Optimized Packet Header Design (OPHD)

4.4.1 Problem

Original CC-AODV uses 32-bit field for boolean flag. 31 wasted bits per packet!

4.4.2 Solution

1. **Use a 2-bit congestion level field** instead of a 32-bit boolean:

- 00 = No congestion ($CL < 0.3$)
- 01 = Low congestion ($0.3 \leq CL < 0.5$)
- 10 = Moderate congestion ($0.5 \leq CL < 0.7$)
- 11 = High congestion ($CL \geq 0.7$)

2. **Add 4-bit hop quality metric:**

- Encodes average queue occupancy along path
- Allows source to select best among multiple RREPs
- Values 0-15 representing 0%-100% occupancy

3. **Result:** Total 6 bits instead of 32 bits

- Memory savings: 81.25% per RREP packet
- More granular congestion information
- Supports path quality comparison

4.5 Modification 4: Multi-Metric Path Selection (MMPS)

4.5.1 Concept

Source node receives multiple RREPs and selects best path based on multiple criteria:

1. Hop count (lower is better)

2. Cumulative congestion level (lower is better)
3. Path quality metric (higher is better)
4. Route freshness (sequence number)

4.5.2 Path Score Formula

$$Score = \alpha \times \frac{1}{HopCount} + \beta \times PathQuality + \gamma \times \frac{1}{CongestionLevel + 1} \quad (2)$$

Where:

- $\alpha = 0.3$ (hop count weight)
- $\beta = 0.4$ (path quality weight)
- $\gamma = 0.3$ (congestion avoidance weight)

Select path with highest score.

4.5.3 Algorithm

Algorithm 2 Multi-Metric Path Selection

```

1: paths  $\leftarrow$  all received RREPs for destination
2: best_path  $\leftarrow$  null
3: best_score  $\leftarrow$  0
4:
5: for each path in paths do
6:   hop_score  $\leftarrow$   $\alpha / path.hop\_count$ 
7:   quality_score  $\leftarrow$   $\beta \times (path.path\_quality / 15)$ 
8:   congestion_score  $\leftarrow$   $\gamma / (path.congestion\_level + 1)$ 
9:
10:  total_score  $\leftarrow$  hop_score + quality_score + congestion_score
11:
12:  if total_score > best_score then
13:    best_score  $\leftarrow$  total_score
14:    best_path  $\leftarrow$  path
15:  end if
16: end for
17:
18: return best_path

```

4.6 Complete ECC-AODV Algorithm Flow

5 Challenges and Mitigation

5.1 Technical Challenges

1. **Challenge:** Balancing multiple metrics in path selection

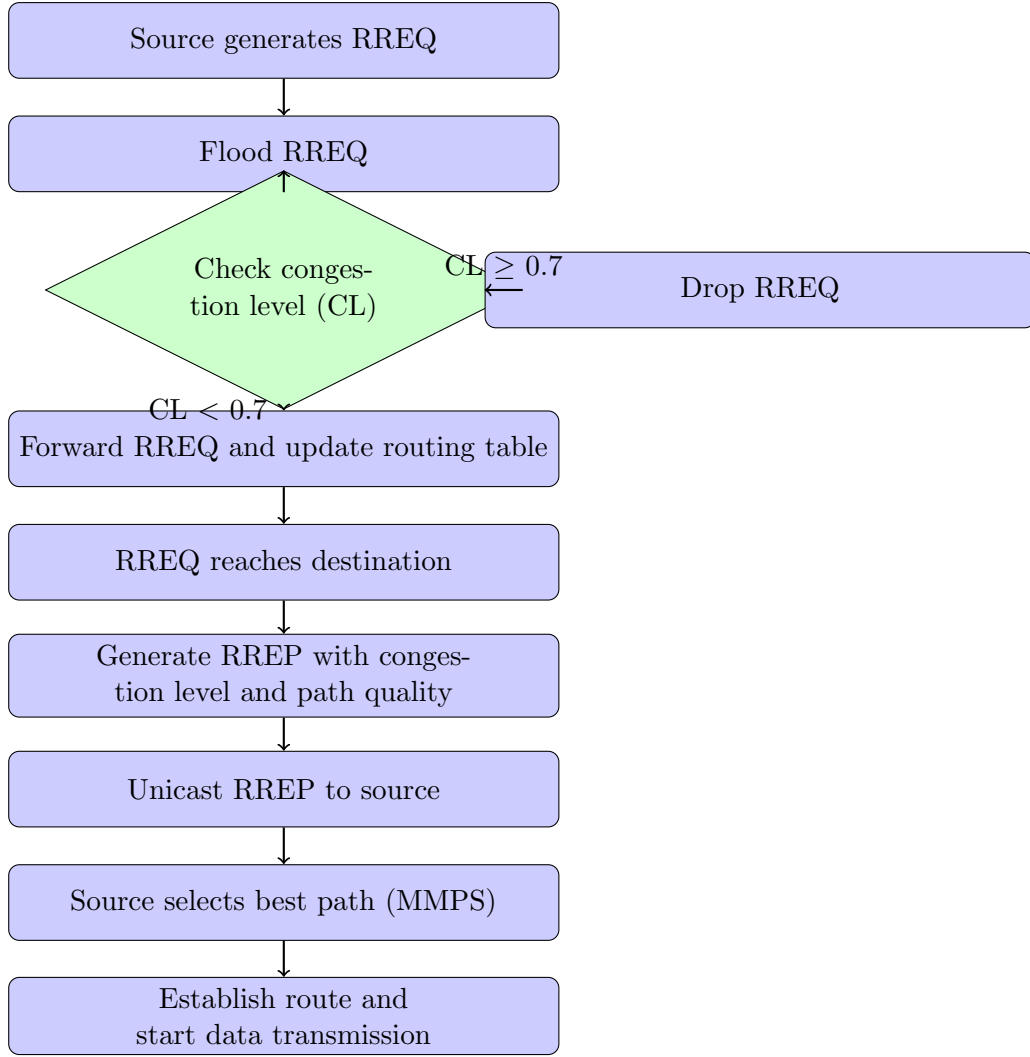


Figure 1: ECC-AODV Protocol Flow

- **Mitigation:** Extensive testing to tune weights (α , β , γ)
 - Use machine learning for weight optimization (future work)
2. **Challenge:** Overhead of queue monitoring
 - **Mitigation:** Periodic sampling instead of continuous monitoring
 - Use lightweight queue statistics already maintained by NS-3
 3. **Challenge:** Increased computation for adaptive threshold
 - **Mitigation:** Update threshold only periodically (every 5s)
 - Use efficient moving average algorithms
 4. **Challenge:** Backward compatibility with vanilla AODV
 - **Mitigation:** Use reserved bits in AODV header
 - Ensure graceful degradation when ECC-AODV nodes communicate with AODV nodes

6 Future Work

6.1 Potential Extensions

1. Machine Learning Integration:

- Use ML to predict congestion before it occurs
- Optimize weight parameters automatically
- Anomaly detection for malicious nodes

2. Security Enhancements:

- Detect nodes falsely reporting congestion
- Implement authentication for routing messages

References

1. Y. Mai, F. M. Rodriguez and N. Wang, "CC-ADOV: An effective multiple paths congestion control AODV," 2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, NV, USA, 2018, pp. 1000-1004. doi: 10.1109/CCWC.2018.8301758